

SISTEMA DE GESTION DE VUELOS

Documento de Planificacion

Migracion al Bus de Eventos con RabbitMQ

Reto #3 - Marketplace del Cliente

Materia: Integracion de Sistemas

Instructora: Maria Paulina Astudillo

Pontificia Universidad Catolica del Ecuador

Junio 2026

1. Introduccion

El presente documento describe el diseño técnico de la Fase 4 del proyecto RedCar, cuyo objetivo es incorporar una arquitectura orientada a eventos (Event-Driven Architecture) mediante el message broker RabbitMQ. Esta fase no reemplaza la arquitectura existente, sino que la complementa: el bus empresarial actual (Middleware.Vuelos, ESB) se mantiene operativo y sin modificaciones para atender la superficie de administración y la integración con el Booking Prototipo, mientras que se introduce un nuevo bus de eventos exclusivo para la superficie del Marketplace MPAS Airways.

El principio rector de la fase es preservar todo lo que ya funciona y reutilizarlo. Las operaciones transaccionales críticas continúan resolviéndose mediante gRPC, garantizando la atomicidad ya implementada, mientras que el desacoplamiento y la asincronía aportados por RabbitMQ se aplican al flujo de reservas iniciado desde el Marketplace.

El documento se organiza en cinco secciones solicitadas: (1) fundamentos teóricos del bus de eventos y de RabbitMQ; (2) criterios técnicos para su utilización; (3) mejores prácticas para migrar del ESB al bus de eventos únicamente en el Marketplace; (4) especificación de los microservicios que deben actualizarse; y (5) el diseño arquitectónico del software, acompañado de sus diagramas.

Directrices del Profesor

- Un GraphQL que englobe todo el marketplace del cliente.
- NO se toca el Bus de Integracion actual. Se crea un Bus de Eventos nuevo y separado.
- GraphQL debe implementarse en el Gateway del Bus de Eventos.
- Todo asincrono, todo desacoplado.
- Paso a paso, uno por uno.
- Primero modificar los MS que van a generar los eventos, luego configurar RabbitMQ, luego el Gateway (donde llegan las peticiones del marketplace), y finalmente probar con Swagger o Postman.
- El Frontend del Marketplace debe migrar a React Native o Flutter.
- INSTALAR DOCKER Y CONFIGURAR EN CADA COMPUTADORA

1. Fundamentos Teóricos del Bus de Eventos y de RabbitMQ

1.1. Qué es un Bus de Eventos

Un bus de eventos (Event Bus) es un patrón de comunicación asíncrona en el cual los componentes de un sistema no se invocan directamente entre sí, sino que publican eventos — hechos que ya han ocurrido— hacia un canal común, y otros componentes se suscriben a aquellos eventos que les resultan relevantes. El emisor del evento desconoce quién lo consumirá y no espera una respuesta inmediata. Esta independencia entre emisor y receptor constituye el desacoplamiento (bajo acoplamiento o low coupling), que es la propiedad fundamental de una arquitectura orientada a eventos.

En una arquitectura de microservicios, el bus de eventos resuelve el problema de las dependencias cruzadas: ningún servicio puede operar de forma totalmente aislada, pues requiere información o acciones de otros servicios. El bus permite que los servicios se mantengan en contacto sin respuestas bloqueadas, empujando mensajes a una cola de forma asíncrona y asegurando que se entreguen al destino correcto.

1.2. Qué es RabbitMQ

RabbitMQ es uno de los message brokers (intermediarios de mensajes) de código abierto más utilizados del mundo. Es el software encargado de definir colas, conectar aplicaciones y aceptar mensajes, habilitando la comunicación asíncrona entre las aplicaciones que producen y las que consumen mensajes. En lugar de comunicarse directamente entre sí, las aplicaciones interactúan con el broker.

Una de sus ventajas centrales es que hace los datos temporalmente persistentes: el broker almacena los mensajes hasta que una aplicación receptora se conecta y los consume. Esto reduce el riesgo de errores cuando alguna parte del sistema está fuera de servicio; si un componente es inalcanzable, los demás continúan interactuando con la cola sin verse bloqueados.

1.3. Conceptos Fundamentales de RabbitMQ

Los siguientes conceptos son la base sobre la cual se diseñará el bus de eventos del Marketplace:

Concepto	Definición
Producer	Aplicación que produce y publica mensajes. En el Marketplace, el componente que recibe la solicitud de reserva.
Consumer	Aplicación que recibe y procesa los mensajes. En RedCar, los microservicios suscritos al bus.
Exchange	Punto de entrada del mensaje al broker. El productor nunca publica directamente a una cola, sino a un exchange, que enruta el mensaje según su tipo y reglas.
Queue (Cola)	Lugar donde se almacenan los mensajes hasta que el consumidor los procesa y confirma.
Binding	Asociación o vínculo configurado entre una cola y un exchange; describe qué cola está interesada en los mensajes de un exchange.
Routing Key	Atributo del mensaje que el exchange examina para decidir hacia qué cola enrutarlo. Actúa como la dirección de destino del mensaje.

Concepto	Definición
Acknowledgment (ack)	Confirmación que el consumidor envía al broker indicando que recibió o procesó el mensaje. El broker solo elimina el mensaje de la cola tras esta confirmación.
Vhost	Host virtual que segrega aplicaciones dentro de un mismo broker, de forma análoga a los esquemas de una base de datos PostgreSQL.
Dead Letter Exchange (DLX)	Exchange al que se redirigen los mensajes que fallan, expiran (TTL) o son rechazados, para su posterior inspección o reproceso sin pérdida de información.

1.4. Tipos de Exchange

RabbitMQ ofrece cuatro tipos de exchange que enrutan los mensajes de forma distinta. La elección del tipo determina cómo se distribuyen los eventos entre las colas:

- **Direct:** entrega el mensaje a la cola cuya binding key coincide exactamente con la routing key del mensaje. Útil para distinguir mensajes mediante un identificador simple.
- **Topic:** realiza una coincidencia por patrones con comodines sobre la routing key, usando el asterisco (*) para una palabra y el numeral (#) para cero o más palabras. Permite que un mismo evento llegue a varias colas según su jerarquía (por ejemplo, marketplace.reserva.confirmada).
- **Fanout:** copia y enruta el mensaje a todas las colas vinculadas, ignorando la routing key. Útil para difundir un mismo evento a múltiples consumidores.
- **Headers:** enruta según los atributos de la cabecera del mensaje en lugar de la routing key.

Para el Marketplace RedCar se adoptará un exchange de tipo **Topic**, por ser el más flexible: permite que un único evento de reserva sea consumido simultáneamente por varios microservicios mediante patrones de enrutamiento jerárquicos.

1.5. Diferencia entre el ESB y el Bus de Eventos

La distinción entre el bus empresarial actual (ESB) y el nuevo bus de eventos es la justificación conceptual de esta fase:

Aspecto	ESB (Middleware.RedCar actual)	Bus de Eventos (RabbitMQ)
Comunicación	Síncrona (petición-respuesta)	Asíncrona (publicación-suscripción)
Acoplamiento	El orquestador conoce a cada microservicio	El productor no conoce a los consumidores
Inteligencia	Centralizada en el orquestador	Distribuida en los consumidores

Aspecto	ESB (Middleware.RedCar actual)	Bus de Eventos (RabbitMQ)
Si un servicio cae	La operación falla o se bloquea	El mensaje permanece en la cola
Resiliencia	El ESB es un punto central	El broker desacopla y persiste

La comunicación HTTP síncrona, aunque parece más sencilla por su naturaleza familiar de petición-respuesta, presenta limitaciones de fondo: acoplamiento fuerte, dificultades en el aislamiento de fallos, rigidez para introducir cambios y tiempos de respuesta prolongados. La adopción de colas de mensajes permite una arquitectura más desacoplada que mejora el aislamiento de fallos y la flexibilidad.

2. Criterios Técnicos para Utilizar el Bus de Eventos y RabbitMQ

No toda operación del Marketplace debe migrarse a eventos. Los siguientes criterios determinan qué operaciones se benefician de la asincronía y cuáles deben permanecer síncronas.

2.1. Criterio 1: Naturaleza de la Operación (Lectura vs. Escritura con Efectos)

El Marketplace presenta dos tipos de operaciones con necesidades opuestas:

- **Lecturas** (consultas): el usuario necesita la respuesta de forma inmediata para visualizar el catálogo. Un mecanismo asíncrono no es adecuado, pues no puede mostrar resultados que llegarán más tarde. Estas operaciones permanecen como peticiones REST síncronas.
- **Escrituras con efectos en cadena** (creación de una reserva): tras iniciar la operación se desencadenan acciones —validación, persistencia transaccional, confirmación, sincronización y auditoría— que se benefician del desacoplamiento. Este es el flujo que se migra a eventos.

2.2. Criterio 2: Atomicidad Transaccional

RabbitMQ no provee transacciones atómicas que abarquen múltiples tablas; esa garantía la aporta la transacción gRPC ya implementada en MS.Reservas, la cual crea de forma indivisible la reserva, los conductores por reserva, los extras por reserva y la factura, respetando la restricción de base de datos que exige la coherencia de los subtotales de extras. Por lo tanto, el criterio es claro: aquello que requiere atomicidad estricta no se migra a eventos; aquello que tolera consistencia eventual sí.

La consecuencia de diseño es que el evento y la transacción gRPC no compiten, sino que se encadenan: el evento dispara el flujo y, dentro del consumidor, la transacción gRPC garantiza la atomicidad. De este modo se reutiliza íntegramente la lógica transaccional existente en lugar de duplicarla.

2.3. Criterio 3: El Flujo de Reserva del Marketplace como Operación Asíncrona

La decisión adoptada para esta fase es que la creación de reservas iniciada desde el Marketplace RedCar se gestione mediante eventos de forma asíncrona. El flujo opera de la siguiente manera: el Marketplace publica un evento de reserva solicitada y responde de inmediato al cliente con un acuse de recibo (HTTP 202 Accepted) y un identificador de seguimiento; el microservicio de Reservas consume ese evento, ejecuta internamente su transacción atómica vía gRPC y publica un evento de reserva confirmada o rechazada; finalmente, el estado de la reserva queda disponible para que el frontend lo consulte.

Esta aproximación implica aceptar la consistencia eventual en la interfaz del Marketplace: la confirmación no es instantánea, por lo que el frontend mostrará un estado intermedio de “reserva en proceso” y revelará el resultado definitivo cuando el evento de confirmación haya sido procesado. Este es el comportamiento esperado y correcto de una arquitectura orientada a eventos.

2.4. Criterio 4: Garantías de Entrega

Dado que las reservas constituyen datos críticos, el diseño prioriza la garantía de entrega sobre el rendimiento máximo. Se aplican las siguientes configuraciones:

- Colas declaradas como **durables** y mensajes marcados como **persistentes**, de modo que sobrevivan a reinicios del broker.
- **Confirmación manual (ack manual)** por parte del consumidor: el mensaje solo se elimina de la cola una vez procesado satisfactoriamente.
- **Dead Letter Exchange (DLX)** asociado a las colas de reservas, para capturar y reprocesar los mensajes que fallen sin pérdida de información.

2.5. Criterio 5: Control de Consumo (Prefetch)

Se configurará un valor de prefetch por consumidor en lugar del comportamiento por defecto (búfer ilimitado). Establecer un límite sobre el número de mensajes no confirmados que el broker entrega a la vez permite distribuir la carga de manera equitativa entre consumidores y mantener la regularidad en la entrega de mensajes.

3. Mejores Prácticas para Migrar del ESB al Bus de Eventos (solo Marketplace)

La migración se realiza exclusivamente sobre la superficie del Marketplace, preservando intactos el ESB para administración y la integración con el Booking Prototipo. Las siguientes prácticas guían el proceso.

3.1. Migración incremental, nunca total

La migración se aborda de forma incremental y por operación, validando cada paso, en lugar de una reescritura completa. El ESB continúa sirviendo a la administración y al Booking mientras el

bus de eventos crece a su lado. Este enfoque gradual es el recomendado para evolucionar de una arquitectura monolítica o centralizada hacia microservicios orientados a eventos sin interrumpir el servicio.

3.2. Preservar y envolver, no destruir

En lugar de eliminar la lógica del Marketplace del Middleware, el nuevo bus la reutiliza. La lógica transaccional de reservas no se reescribe: se envuelve con un consumidor de eventos que la invoca. De esta manera, el commit atómico vía gRPC permanece idéntico y únicamente cambia su disparador, que pasa de ser una llamada directa a ser el consumo de un evento.

3.3. Reutilizar clientes y contratos existentes

El sistema ya dispone de clientes HTTP tipados hacia los microservicios, el contrato gRPC definido en el archivo `.proto` y los DTOs correspondientes. Los componentes productores y consumidores del Marketplace reaprovechan estos elementos en lugar de reinventarlos, lo que reduce el riesgo de introducir inconsistencias.

3.4. Exchange de tipo Topic con claves de enrutamiento jerárquicas

Se define un único exchange de tipo Topic para el Marketplace, con claves de enrutamiento jerárquicas tales como `marketplace.reserva.solicitada` y `marketplace.reserva.confirmada`. Así, un consumidor de auditoría puede suscribirse a todos los eventos mediante el patrón `marketplace.#` mientras que un consumidor específico escucha únicamente `marketplace.reserva.*`.

3.5. Aislamiento mediante Vhost dedicado

Todo el tráfico del Marketplace se aísla en un host virtual propio, separado de cualquier otro uso del broker. Esto segrega usuarios, exchanges y colas, de forma análoga a separar bases de datos en un mismo servidor.

3.6. Idempotencia en los consumidores

Con un esquema de entrega “al menos una vez”, un consumidor puede recibir el mismo evento más de una vez. Por ello, los manejadores se diseñan de forma idempotente: reprocesar el mismo evento no debe producir efectos duplicados (por ejemplo, verificando si una reserva o un registro de auditoría ya existe antes de crearlo).

3.7. Dead Letter Exchange desde el inicio

Cada cola de reservas se acompaña de su cola de mensajes muertos (Dead Letter Queue) desde el primer momento, lo que permite inspeccionar y reprocesar manualmente los eventos fallidos sin que se pierda ninguna reserva.

3.8. Preservación del contrato externo del Booking

La integración con el Booking Prototipo no se modifica. Sus endpoints continúan respondiendo de forma síncrona vía gRPC, y las reglas de negocio vigentes —la identificación del canal de reserva y el cálculo automático del IVA del 15%— se mantienen sin cambios. El Booking no debe percibir ninguna alteración interna.

4. Microservicios que Deben Actualizarse para el Bus de Eventos del Marketplace

Estado Actual del Sistema

El sistema ya cuenta con todos los microservicios implementados, desplegados en Azure App Service y funcionando correctamente de forma sincronica a traves del Bus de Integracion REST.

Microservicios existentes

Microservicio	Responsabilidad
MS Seguridad	Login, JWT, roles, usuarios
MS Geografia	Países y ciudades (catalogo)
MS Aeropuertos	CRUD aeropuertos, IATA/ICAO
MS Vuelos	Vuelos, escalas, asientos, booking
MS Clientes	Clientes y pasajeros
MS ReservasF	Reservas, facturas, boletos, equipaje
Bus de Integracion	Orquestador REST + portal cliente

Flujo actual (sincronico REST)

El flujo actual del marketplace funciona de la siguiente forma:

- Frontend -> Bus de Integracion (REST) -> MS Vuelos -> respuesta inmediata
- Frontend -> Bus de Integracion (REST) -> MS Clientes -> respuesta inmediata
- Frontend -> Bus de Integracion (REST) -> MS ReservasF -> respuesta inmediata

Problema critico: cuando el cliente paga, ReservasF llama a MS Vuelos por gRPC para bloquear asientos dentro de la misma operacion. Si esa llamada falla, puede quedar una reserva sin asiento bloqueado o un asiento bloqueado sin reserva emitida.

gRPC ya implementado

La variante productiva del sistema usa gRPC entre MS ReservasF y MS Vuelos para operaciones criticas:

Operacion	Protocolo	RPC
Crear reserva (validar vuelo/asiento)	gRPC	GetVuelo, ValidarAsiento
Pagar reserva (bloquear asiento)	gRPC	BloquearAsiento
Cancelar reserva emitida (liberar)	REST via Bus	PATCH disponibilidad asiento

Arquitectura Propuesta - Bus de Eventos

Principios de diseno

- El Bus de Integracion (Middleware) NO se modifica. Sigue funcionando igual para todo lo administrativo y el booking REST.
- Se crea un NUEVO proyecto separado: el Bus de Eventos del Marketplace.
- El Bus de Eventos expone GraphQL como unico punto de entrada para el frontend del marketplace.
- Los microservicios existentes solo reciben cambios minimos: agregar consumers y publishers para responder a eventos.
- Nada de logica de negocio se toca. Solo se anade la capa de mensajeria.

Diagrama de capas

La arquitectura queda dividida en dos capas paralelas e independientes:

Capa	Descripcion	Punto de entrada	Protocolo
Bus de Integracion (existente)	Todo lo administrativo: CRUD vuelos, aeropuertos, clientes, reservas admin, portal cliente REST	REST HTTP + JWT	HTTP/REST + gRPC interno
Bus de Eventos (NUEVO)	Solo el flujo del marketplace: buscar, seleccionar, pagar, reservar, facturar, emitir boleto	GraphQL (Hot Chocolate)	RabbitMQ asincrono

Microservicios que se actualizan

Solo tres microservicios reciben cambios, y unicamente para agregar consumers y publishers:

Microservicio	Rol en el Bus de Eventos	Nivel de cambio
MS Vuelos	Consumer: escucha VueloSeleccionado y BoletoEmitido. Publisher: publica AsientoPreReservado.	Medio
MS ReservasF	Consumer: ReservaSolicitada, AsientoPreReservado, EquipajeAgregado. Publisher: ReservaCreada, FacturaGenerada, BoletoEmitido.	Alto
MS Clientes	Consumer: PasajerosRegistrados. Publisher: PasajerosValidados.	Bajo
MS Seguridad	Sin cambios.	Ninguno
MS Geografia	Sin cambios.	Ninguno

Microservicio	Rol en el Bus de Eventos	Nivel de cambio
MS Aeropuertos	Sin cambios. Busqueda sigue por REST.	Ninguno

Flujo de Eventos del Marketplace

El flujo del cliente en el marketplace queda dividido en dos partes: lo que sigue siendo sincronico (el cliente necesita respuesta inmediata) y lo que pasa por el Bus de Eventos (procesos que pueden ser asincronos).

Flujo sincronico REST (no cambia)

Accion del cliente	Endpoint	Donde va
Buscar vuelos	GET /api/v1/booking/vuelos/buscar	Bus REST -> MS Vuelos
Ver detalle de vuelo	GET /api/v1/booking/vuelos/{id}	Bus REST -> MS Vuelos
Ver asientos disponibles	GET /api/v1/booking/vuelos/{id}/asientos	Bus REST -> MS Vuelos
Buscar aeropuertos	GET /api/v1/booking/aeropuertos	Bus REST -> MS Aeropuertos
Login / registro	POST /api/v1/auth/login	Bus REST -> MS Seguridad

Flujo asincrono por eventos (nuevo)

Paso	Evento publicado	Exchange	Consumer
1. Cliente selecciona vuelo y asiento	VueloSeleccionado	vuelos.exchange	MS Vuelos -> publica AsientoPreReservado (15 min)
2. Cliente registra pasajeros	PasajerosRegistrados	clientes.exchange	MS Clientes -> publica PasajerosValidados
3. Cliente agrega equipaje	EquipajeAgregado	reservas.exchange	MS ReservasF -> encola equipaje
4. Cliente confirma reserva	ReservaSolicitada	reservas.exchange	MS ReservasF -> crea reserva PEN -> publica ReservaCreada
5. Generacion de factura	ReservaCreada	facturacion.exchange	MS ReservasF -> genera factura -> publica FacturaGenerada
6. Emision de boleto	FacturaGenerada	facturacion.exchange	MS ReservasF -> emite boleto -> publica BoletoEmitido
7. Bloqueo definitivo de asiento	BoletoEmitido	facturacion.exchange	MS Vuelos -> bloquea asiento definitivamente (EMI)

Exchanges y Colas en RabbitMQ

Exchange	Tipo	Cola	Consumer
vuelos.exchange	Topic	reservas.vuelo-seleccionado.queue	MS Vuelos
clientes.exchange	Topic	reservas.pasajeros-registrados.queue	MS Clientes
reservas.exchange	Topic	reservas.reserva-solicitada.queue	MS ReservasF
reservas.exchange	Topic	reservas.equipaje-agregado.queue	MS ReservasF
facturacion.exchange	Topic	vuelos.boleto-emitado.queue	MS Vuelos
facturacion.exchange	Topic	notificaciones.reserva-creada.queue	Futuro: notificaciones
facturacion.exchange	Topic	auditoria.reserva-creada.queue	Futuro: auditoria

Cada cola principal debe tener su correspondiente Dead Letter Queue (DLQ) para mensajes que fallan repetidamente. Por ejemplo: reservas.vuelo-seleccionado.queue.dlq

Nuevo Proyecto: Bus de Eventos del Marketplace

Este es el proyecto central del Reto #3. Se crea desde cero como un proyecto .NET 10 separado, independiente del Bus de Integracion existente.

Responsabilidades

- Exponer GraphQL (Hot Chocolate) como unico punto de entrada para el frontend del marketplace.
- Publicar eventos a RabbitMQ cuando el cliente realiza acciones (seleccionar vuelo, registrar pasajeros, confirmar reserva).
- Escuchar eventos de respuesta de los microservicios para notificar al frontend el estado de la saga.
- Gestionar el estado del flujo de una reserva (para que el frontend pueda hacer polling o suscribirse).

Pasos del Reto #3 (orden del profesor)

El profesor indico explicitamente el siguiente orden de implementacion:

Paso 1 - Instalar Docker en cada computadora

Paso 2 - Modificar los microservicios para eventos

Este es el primer paso de codigo. Se modifica MS Vuelos, MS ReservasF y MS Clientes para que puedan publicar y consumir eventos. La logica de negocio no se toca.

Paso 3 - Configurar el Gateway con GraphQL

Paso 4 - Frontend del Marketplace

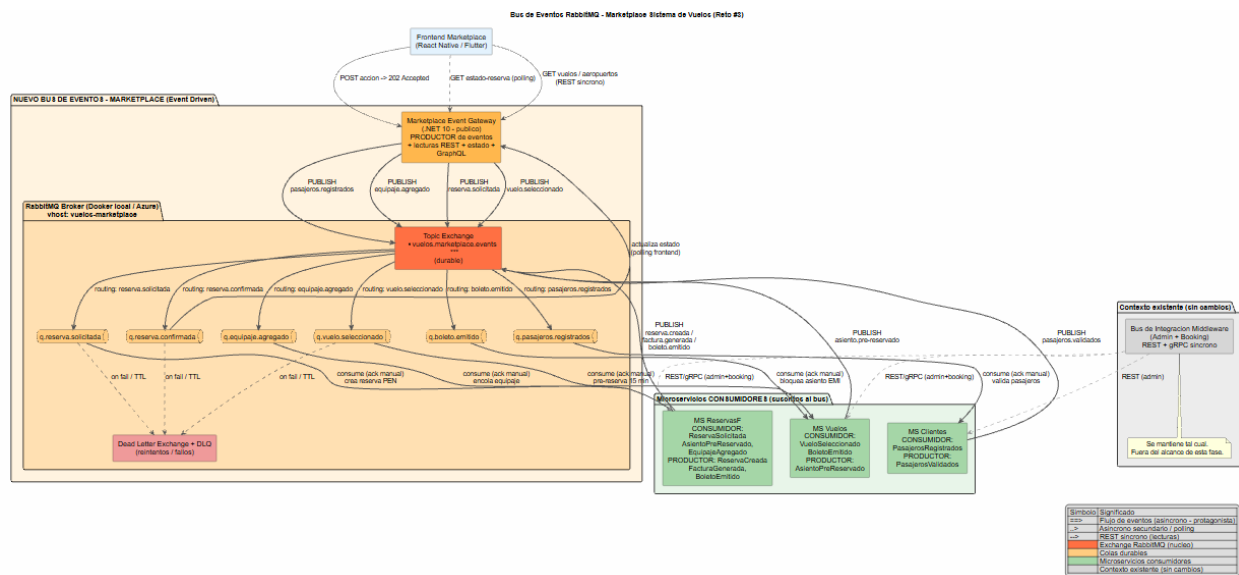
- Decidir entre React Native o Flutter (por definir con el equipo)
- Migrar el flujo del marketplace actual a la aplicacion movil
- Conectar al Bus de Eventos por GraphQL en lugar del Bus de Integracion REST

LA PARTE ADMINISTRATIVA NO SE TOCA SOLO LO QUE HACE EL CLIENTE CUANDO ENTRA A LA INTERFAZ DE VUELOS, SOLO ESO ES LO QUE SE TOPA Y SE MIGRA. SE TOMA EN CUENTA SOLO LA CARACTERISTICA DE MENSAJES

MI PROFESOR DIJO EL OPEN API NO CAMBIA SOLO SE AGREGA UN MESSAGE

11. Codigo PlantUML - Diagrama de Arquitectura

El siguiente codigo PlantUML representa la arquitectura del Bus de Eventos del Marketplace del Sistema de Gestion de Vuelos. Pegarlo en <https://www.plantuml.com/plantuml/uml> para generar el diagrama.



```

@startuml
SistemaVuelos_EventBus_Marketplace
title Bus de Eventos RabbitMQ - Marketplace Sistema de Vuelos (Reto #3)
skinparam componentStyle rectangle
skinparam shadowing false
skinparam defaultTextAlignment center
skinparam ArrowColor #555555
skinparam wrapWidth 200

' ===== CONTEXTO EXISTENTE (sin cambios) =====
package "Contexto existente (sin cambios)" #ECECEC {
    [Bus de Integracion Middleware\n(Admin + Booking)\nREST + gRPC sincrónico] as BUS
    #D6D6D6

    note bottom of BUS
        Se mantiene tal cual.
        Fuera del alcance de esta fase.
    end note
}

```

```
' ===== FRONTEND =====
[Frontend Marketplace\n(React Native / Flutter)] as FMkt #E3F2FD

' ===== NUEVO BUS DE EVENTOS =====
package "NUEVO BUS DE EVENTOS - MARKETPLACE (Event Driven)" #FFF3E0 {

    [Marketplace Event Gateway\n(.NET 10 - publico)\nPRODUCTOR de eventos\n+ lecturas
    REST + estado + GraphQL] as Gateway #FFB74D

    package "RabbitMQ Broker (Docker local / Azure)\nvhost: vuelos-marketplace" #FFE0B2
    {
        [Topic Exchange\n*** vuelos.marketplace.events ***\n(durable)] as EX #FF7043

        queue "q.vuelo.seleccionado" as QVS #FFCC80
        queue "q.pasajeros.registrados" as QPR #FFCC80
        queue "q.equipaje.agregado" as QEA #FFCC80
        queue "q.reserva.solicitada" as QRS #FFCC80
        queue "q.reserva.confirmada" as QRC #FFCC80
        queue "q.boleto.emitido" as QBE #FFCC80

        [Dead Letter Exchange + DLQ\n(reintentos / fallos)] as DLX #EF9A9A
    }
}

' ===== MICROSERVICIOS CONSUMIDORES =====
package "Microservicios CONSUMIDORES (suscritos al bus)" #E8F5E9 {
    [MS Vuelos\nCONSUMIDOR: VueloSeleccionado\nBoletoEmitido\nPRODUCTOR:
    AsientoPreReservado] as MSV #A5D6A7

    [MS ReservasF\nCONSUMIDOR: ReservaSolicitada\nAsientoPreReservado,
    EquipajeAgregado\nPRODUCTOR: ReservaCreada\nFacturaGenerada, BoletoEmitido] as MSR
    #A5D6A7

    [MS Clientes\nCONSUMIDOR: PasajerosRegistrados\nPRODUCTOR: PasajerosValidados] as
    MSC #A5D6A7
}

' ===== RELACIONES FRONTEND -> GATEWAY =====
FMkt --> Gateway : POST accion -> 202 Accepted
FMkt ..> Gateway : GET estado-reserva (polling)
FMkt --> Gateway : GET vuelos / aeropuertos\n(REST sincrono)

' ===== GATEWAY -> EXCHANGES =====
Gateway ==> EX : PUBLISH\nvuelo.seleccionado
Gateway ==> EX : PUBLISH\npasajeros.registrados
Gateway ==> EX : PUBLISH\nequipaje.agregado
Gateway ==> EX : PUBLISH\nreserva.solicitada
```



```
' ===== ROUTING EXCHANGE -> COLAS =====
EX ==> QVS : routing: vuelo.seleccionado
EX ==> QPR : routing: pasajeros.registrados
EX ==> QEA : routing: equipaje.agregado
EX ==> QRS : routing: reserva.solicitada
EX ==> QRC : routing: reserva.confirmada
EX ==> QBE : routing: boleto.emitido

' ===== COLAS -> CONSUMIDORES =====
QVS ==> MSV : consume (ack manual)\npre-reserva 15 min
QPR ==> MSC : consume (ack manual)\nvalida pasajeros
QEA ==> MSR : consume (ack manual)\nencola equipaje
QRS ==> MSR : consume (ack manual)\ncrea reserva PEN

' ===== RESPUESTAS MS -> EXCHANGE =====
MSV ==> EX : PUBLISH\nasiento.pre-reservado
MSR ==> EX : PUBLISH\nreserva.creada /\nfactura.generada /\nboleto.emitido
MSC ==> EX : PUBLISH\npasajeros.validados

' ===== COLAS DE RESPUESTA -> GATEWAY =====
QRC ==> Gateway : actualiza estado\n(polling frontend)
QBE ==> MSV : consume (ack manual)\nbloquea asiento EMI

' ===== DLQ =====
QVS ..> DLX : on fail / TTL
QRS ..> DLX : on fail / TTL
QRC ..> DLX : on fail / TTL

' ===== COEXISTENCIA CON BUS EXISTENTE =====
BUS ..> MSV #999999 : REST/gRPC (admin+booking)
BUS ..> MSR #999999 : REST/gRPC (admin+booking)
BUS ..> MSC #999999 : REST (admin)
```

legend right

Simbolo	Significado
==>	Flujo de eventos (asincrono - protagonista)
..>	Asincrono secundario / polling
-->	REST sincrono (lecturas)
<#FF7043>	Exchange RabbitMQ (nucleo)
<#FFCC80>	Colas durables
<#A5D6A7>	Microservicios consumidores
<#D6D6D6>	Contexto existente (sin cambios)

endlegend

@endum1

Plan de Implementación

La implementación del bus de eventos se llevará a cabo de forma secuencial e incremental, validando cada etapa antes de avanzar a la siguiente y preservando en todo momento los componentes que ya operan correctamente. El orden de implementación definido es el siguiente:

1. Modificación de los microservicios para soportar eventos

El primer paso consiste en habilitar la capacidad de consumo y publicación de eventos en los microservicios que lo requieren. MS.Reservas se configura como consumidor del evento de reserva solicitada y productor del evento de confirmación, reutilizando su transacción atómica vía. Esta etapa se aborda primero porque define los contratos de evento sobre los cuales se construyen las etapas posteriores.

2. Configuración de RabbitMQ (entorno local y posterior despliegue en Oracle Cloud)

A continuación se configura el broker RabbitMQ, comenzando en un entorno local mediante Docker sobre Windows. Esto permite validar la declaración del exchange de tipo Topic, las colas durables, los bindings, el host virtual dedicado y el Dead Letter Exchange en un entorno controlado y de bajo riesgo.

6.3. Configuración del Marketplace Event Gateway con GraphQL

Con los microservicios preparados y el broker operativo, se implementa el Marketplace Event Gateway exponiendo una API GraphQL como punto único de entrada para el frontend del Marketplace. El Gateway resuelve las consultas síncronas mediante GraphQL, reutilizando los clientes existentes hacia los microservicios, y traduce las operaciones de escritura —en particular la creación de reservas— en la publicación de eventos hacia RabbitMQ. También expone el mecanismo de consulta de estado de la reserva que sustenta la consistencia eventual del flujo asíncrono.

6.4. Migración del frontend del Marketplace a React Native (aplicación móvil)

Finalmente, se migra exclusivamente el frontend del Marketplace —no el módulo administrativo— hacia React Native, con el objetivo de convertirlo en una aplicación móvil. Esta nueva interfaz consume la API GraphQL del Gateway, gestiona el estado intermedio de "reserva en proceso" propio de la arquitectura orientada a eventos y muestra el resultado definitivo una vez procesado el evento de confirmación. El módulo administrativo permanece sin cambios en la página web anterior, continuando su operación a través del ESB existente.